1. Pętla for w różnych wariantach

Standardowa pętla po liście elementów

Pozwala na iterowanie po zdefiniowanej liście wartości tekstowych:

Bash

```
#!/bin/bash
for test in Nevada "New Hampshire" "New Mexico" "New York"
do
	echo "Teraz jadę do $test"
done
```[cite: 64]

Odczytywanie zawartości pliku linia po linii
Można wykorzystać polecenie `cat` wewnątrz podstawienia polecenia do iteracji po elementach z zewnętrznego pliku tekstowego[cite: 64]:
```bash
#!/bin/bash
file="states"
for state in $(cat $file)
do
	echo "Visit beautiful $state"
done
```[cite: 64]

Iteracja po plikach w katalogu i testowanie ich typów
Pętle doskonale sprawdzają się przy operacjach na plikach, łącząc się z testami takimi jak `-d` (czy katalog) lub `-f` (czy plik zwykły)[cite: 64, 65]:
```bash
#!/bin/bash
for file in /home/rich/test/*
do
	if [ -d "$file" ]
	then
		echo "$file is a directory"
	elif [ -f "$file" ]
	then
		echo "$file is a file"
	fi
done
```[cite: 64, 65]

Pętla `for` w stylu języka C
Bash umożliwia stosowanie tradycyjnej składni znanej z języka C, co ułatwia sterowanie licznikiem[cite: 65]:
```bash
#!/bin/bash
for (( i=1; i<=10; i++ ))
do
	echo "The next number is $i"
done
```[cite: 65]

Możliwa jest również obsługa **wielu zmiennych** jednocześnie (np. jedna inkrementowana, druga dekrementowana)[cite: 65, 66]:
```bash
#!/bin/bash
for (( a=1, b=10; a<=10; a++, b-- ))
do
	echo "$a-$b"
done
```[cite: 66]

---

2. Zagnieżdżanie pętli
Pętle dowolnego typu można umieszczać jedne w drugich (np. pętle `for` wewnątrz `for`[cite: 68], `for` wewnątrz `while`[cite: 68, 69], czy `until` z `while`[cite: 69]).

Przykład pętli `for` zagnieżdżonej w `while`:
```bash
#!/bin/bash
var1=5
while [ $var1 -ge 0 ]
do
	echo "Outer loop: $var1"
	for (( var2 = 1; $var2 < 3; var2++ ))
	do
		var3=$(( $var1 * $var2 ))
		echo " Inner loop: $var1 * $var2 = $var3"
	done
	var1=$(( $var1 - 1 ))
done
```[cite: 68, 69]

---

3. Przerywanie i kontynuowanie pętli (`break` oraz `continue`)

* **`break`** – natychmiast przerywa działanie pętli[cite: 70]. W przypadku pętli zagnieżdżonych można podać argument określający, o ile poziomów w górę ma nastąpić przerwanie (np. `break 2`)[cite: 71, 72].
  ```bash
  for (( a=1; a<4; a++ ))
  do
  	for (( b=1; b<100; b++ ))
  	do
  		if [ $b -gt 4 ]
  		then
  			break 2  # Przerywa obie pętle
  		fi
  	done
  done
  ```[cite: 71, 72]

* **`continue`** – przerywa bieżącą iterację i przechodzi do kolejnego obiegu pętli[cite: 72]. Podobnie jak `break`, `continue 2` pozwala kontynuować powiązaną pętlę zewnętrzną[cite: 73, 74].

---

4. Przekierowanie strumieni i potoki w pętlach

* **Zapis całego wyjścia pętli do pliku:**
  Można przekierować wynik działania całej konstrukcji `for` na jej końcu[cite: 74]:
  ```bash
  for (( a=1; a<10; a++ ))
  do
        echo "The number is $a"
  done > test23.txt
  ```[cite: 74]

* **Przesyłanie wyjścia pętli przez potok (`pipe`):**
  Wyniki generowane w pętli można przekazać bezpośrednio do innego polecenia (np. `sort`)[cite: 74]:
  ```bash
  for state in "North Dakota" Connecticut Illinois Alabama Tennessee
  do
        echo "$state is the next place to go"
  done | sort
  ```[cite: 74]